

提示 9-17: 在递推法中，如果计算顺序很特殊，而且计算新状态所用到的原状态不多，可以尝试用滚动数组减少内存开销。

滚动数组虽好，但也存在一些不尽如人意的地方，例如，打印方案较困难。当动态规划结束之后，只有最后一个阶段的状态值，而没有前面的值。不过这也不能完全归咎于滚动数组，规划方向也有一定责任——即使用二维数组，打印方案也不是特别方便。事实上，对于“前 i 个物品”这样的规划方向，只能用逆向的打印方案，而且还不能保证它的字典序最小（字典序比较是从前往后的）。

提示 9-18: 在使用滚动数组后，解的打印变得困难了，所以在需要打印方案甚至要求字典序最小方案的场合，应慎用滚动数组。

例题 9-5 劲歌金曲 (Jin Ge Jin Qu [h]ao, Rujia Liu's Present 6, UVa 12563)

如果问一个麦霸：“你在 KTV 里必唱的曲目有哪些？”得到的答案通常都会包含一首“神曲”：古巨基的《劲歌金曲》。为什么呢？一般来说，KTV 不会在“时间到”的时候鲁莽地把正在唱的歌切掉，而是会等它放完。例如，在还有 15 秒时再唱一首 2 分钟的歌，则实际上多唱了 105 秒。但是融合了 37 首歌曲的《劲歌金曲》长达 11 分 18 秒^①，如果唱这首，相当于多唱了 663 秒！

假定你正在唱 KTV，还剩 t 秒时间。你决定接下来只唱你最爱的 n 首歌（不含《劲歌金曲》）中的一些，在时间结束之前再唱一个《劲歌金曲》，使得唱的总曲目尽量多（包含《劲歌金曲》），在此前提下尽量晚的离开 KTV。

输入 n ($n \leq 50$)， t ($t \leq 10^9$) 和每首歌的长度（保证不超过 3 分钟^②），输出唱的总曲目以及时间总长度。输入保证所有 $n+1$ 首曲子的总长度严格大于 t 。

【分析】

虽说 $t \leq 10^9$ ，但由于所有 $n+1$ 首曲子的总长度严格大于 t ，实际上 t 不会超过 $180n+678$ 。这样就可以转化为 0-1 背包问题了。细节留给读者思考。

9.4 更多经典模型

本节介绍一些常见结构中的动态规划，序列、表达式、凸多边形和树。尽管它们的形式和解法千差万别，但都用到了动态规划的思想：从复杂的题目背景中抽象出状态表示，然后设计它们之间的转移。

9.4.1 线性结构上的动态规划

最长上升子序列问题 (LIS)。 给定 n 个整数 $A_1, A_2, \dots, A_n$ ，按从左到右的顺序选出尽量多的整数，组成一个上升子序列（子序列可以理解为：删除 0 个或多个数，其他数的顺序

^① 还有《劲歌金曲 2》和《劲歌金曲 3》，但本题不予考虑。

^② 显然大多数歌的长度都大于 3 分钟，但是 KTV 可以“切歌”，因此这里的“长度”实际上是指“想唱的时间长度”。

不变)。例如序列 1, 6, 2, 3, 7, 5, 可以选出上升子序列 1, 2, 3, 5, 也可以选出 1, 6, 7, 但前者更长。选出的上升子序列中相邻元素不能相等。

【分析】

设 $d(i)$ 为以 i 结尾的最长上升子序列的长度, 则 $d(i) = \max\{0, d(j) | j < i, A_j < A_i\} + 1$, 最终答案是 $\max\{d(i)\}$ 。如果 LIS 中的相邻元素可以相等, 把小于号改成小于等于号即可。上述算法的时间复杂度为 $O(n^2)$ 。《算法竞赛入门经典》中介绍了一种方法把它优化到 $O(n \log n)$, 有兴趣的读者可以自行阅读。

最长公共子序列问题 (LCS)。给两个子序列 A 和 B, 如图 9-7 所示。求长度最大的公共子序列。例如 1, 5, 2, 6, 8, 7 和 2, 3, 5, 6, 9, 8, 4 的最长公共子序列为 5, 6, 8 (另一个解是 2, 6, 8)。

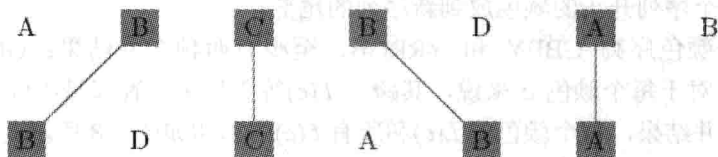


图 9-7 子序列 A 和 B

【分析】

设 $d(i, j)$ 为 $A_1, A_2, \dots, A_i$ 和 $B_1, B_2, \dots, B_j$ 的 LCS 长度, 则当 $A[i] = B[j]$ 时 $d(i, j) = d(i-1, j-1) + 1$, 否则 $d(i, j) = \max\{d(i-1, j), d(i, j-1)\}$, 时间复杂度为 $O(nm)$, 其中 n 和 m 分别是序列 A 和 B 的长度。

例题 9-6 照明系统设计 (Lighting System Design, UVa 11400)

你的任务是设计一个照明系统。一共有 n ($n \leq 1000$) 种灯泡可供选择, 不同种类的灯泡必须用不同的电源, 但同一种灯泡可以共用一个电源。每种灯泡用 4 个数值表示: 电压值 V ($V \leq 132000$), 电源费用 K ($K \leq 1000$), 每个灯泡的费用 C ($C \leq 10$) 和所需灯泡的数量 L ($1 \leq L \leq 100$)。

假定通过所有灯泡的电流都相同, 因此电压高的灯泡功率也更大。为了省钱, 可以把一些灯泡换成电压更高的另一种灯泡以节省电源的钱 (但不能换成电压更低的灯泡)。你的任务是计算出最优方案的费用。

【分析】

首先可以得到一个结论: 每种电压的灯泡要么全换, 要么全不换。因为如果只换部分灯泡, 如 $V=100$ 有两个灯泡, 把其中一个换成 $V=200$ 的, 另一个不变, 则 $V=100$ 和 $V=200$ 两种电源都需要, 不划算 (若一个都不换则只需要 $V=100$ 一种电源)。

先把灯泡按照电压从小到大排序。设 $s[i]$ 为前 i 种灯泡的总数量 (即 L 值之和), $d[i]$ 为灯泡 1~ i 的最小开销, 则 $d[i] = \min\{d[j] + (s[i] - s[j]) * c[i] + k[i]\}$, 表示前 j 个先用最优方案买, 然后第 $j+1$ ~ i 个都用第 i 号的电源。答案为 $d[n]$ 。

例题 9-7 划分成回文串 (Partitioning by Palindromes, UVa 11584)

输入一个由小写字母组成的字符串, 你的任务是把它划分成尽量少的回文串。例如, racecar 本身就是回文串; fastcar 只能分成 7 个单字母的回文串, aaadbccb 最少分成 3 个回

文串: aaa, d, bccb。字符串长度不超过 1000。

【分析】

$d[i]$ 为字符 $0 \sim i$ 划分成的最小回文串的个数, 则 $d[i] = \min\{d[j] + 1 \mid s[j+1 \sim i] \text{ 是回文串}\}$ 。注意频繁的要判断回文串。状态 $O(n)$ 个, 决策 $O(n)$ 个, 如果每次转移都需要 $O(n)$ 时间判断, 总时间复杂度会达到 $O(n^3)$ 。

可以先用 $O(n^2)$ 时间预处理 $s[i..j]$ 是否为回文串。方法是枚举中心, 然后不断向左右延伸并且标记当前子串是回文串, 直到延伸的左右字符不同为止。这样一来, 每次转移的时间降为了 $O(1)$, 总时间复杂度为 $O(n^2)$ 。

例题 9-8 颜色的长度 (Color Length, ACM/ICPC Daejeon 2011, UVa1625)

输入两个长度分别为 n 和 m ($n, m \leq 5000$) 的颜色序列, 要求按顺序合并成同一个序列, 即每次可以把一个序列开头的颜色放到新序列的尾部。

例如, 两个颜色序列 GBBY 和 YRRGB, 至少有两种合并结果: GBYBRYRGB 和 YRRGGBBYB。对于每个颜色 c 来说, 其跨度 $L(c)$ 等于最大位置和最小位置之差。例如, 对于上面两种合并结果, 每个颜色的 $L(c)$ 和所有 $L(c)$ 的总和如图 9-8 所示。

Color	G	Y	B	R	Sum
$L(c)$: Scenario 1	7	3	7	2	19
$L(c)$: Scenario 2	1	7	3	1	12

图 9-8 每个颜色的 $L(c)$ 和 $L(c)$ 的总和

你的任务是找一种合并方式, 使得所有 $L(c)$ 的总和最小。

【分析】

根据前面的经验, 可以设 $d(i, j)$ 表示两个序列已经分别移走了 i 和 j 个元素, 还需要多少费用。等一下! 什么叫“还需要多少费用”呢? 本题的指标函数 (即需要最小化的函数) 比较复杂。当某颜色第一次出现在最终序列中时, 并不知道它什么时候会结束; 而某个颜色的最后一个元素已经移到最终序列里时, 又“忘记”了它是什么时候第一次出现的。

怎么办呢? 如果记录每个颜色的第一次出现位置, 状态会变得很复杂, 时间也无法承受, 所以只能把在指标函数的“计算方式”上想办法: 不是等到一个颜色全部移完之后再算, 而是每次累加。换句话说, 当把一个颜色移到最终序列前, 需要把所有“已经出现但还没结束”的颜色的 $L(c)$ 值加 1。更进一步地, 因为并不关心每个颜色的 $L(c)$, 所以只需要知道有多少种颜色已经开始但尚未结束。

例如, 序列 GBBY 和 YRRGB, 分别已经移走了 1 个和 3 个元素 (例如, 已经合并成了 YRRG)。下次再从序列 2 移走一个元素 (即 G) 时, Y 和 G 需要加 1。下次再从序列 1 移走一个元素 (它是 B) 时, 只有 Y 需要加 1 (因为 G 已经结束)。

这样, 可以事先算出每个颜色在两个序列中的开始和结束位置, 就可以在动态规划时在 $O(1)$ 时间内计算出状态 $d(i, j)$ 中“有多少个颜色已经出现但尚未结束”, 从而在 $O(1)$ 时间内完成状态转移。状态总是为 $O(nm)$ 个, 总时间复杂度也是 $O(nm)$ 。

最优矩阵链乘。 一个 $n \times m$ 矩阵由 n 行 m 列共 $n \times m$ 个数排列而成。两个矩阵 A 和 B 可以相乘当且仅当 A 的列数等于 B 的行数。一个 $n \times m$ 的矩阵乘以一个 $m \times p$ 的矩阵等于一

个 $n \times p$ 的矩阵, 运算量为 mnp 。

矩阵乘法不满足分配律, 但满足结合律, 因此 $A \times B \times C$ 既可以按顺序 $(A \times B) \times C$ 进行, 也可以按 $A \times (B \times C)$ 进行。假设 A 、 B 、 C 分别是 2×3 , 3×4 和 4×5 的, 则 $(A \times B) \times C$ 的运算量为 $2 \times 3 \times 4 + 2 \times 4 \times 5 = 64$, $A \times (B \times C)$ 的运算量为 $3 \times 4 \times 5 + 2 \times 3 \times 5 = 90$ 。显然第一种顺序节省运算量。

给出 n 个矩阵组成的序列, 设计一种方法把它们依次乘起来, 使得总的运算量尽量小。假设第 i 个矩阵 A_i 是 $p_{i-1} \times p_i$ 的。

【分析】

本题任务是设计一个表达式。在整个表达式中, 一定有一个“最后一次乘法”。假设它是第 k 个乘号, 则在此之前已经算出了 $P = A_1 \times A_2 \times \cdots \times A_k$ 和 $Q = A_{k+1} \times A_{k+2} \times \cdots \times A_n$ 。由于 P 和 Q 的计算过程互不相干, 而且无论按照怎样的顺序, P 和 Q 的值都不会发生改变, 因此只需分别让 P 和 Q 按照最优方案计算(最优子结构!)即可。为了计算 P 的最优方案, 还需要继续枚举 $P = A_1 \times A_2 \times \cdots \times A_k$ 的“最后一次乘法”, 把它分成两部分。不难发现, 无论怎么分, 在任意时候, 需要处理的子问题都形如“把 $A_i, A_{i+1}, \dots, A_j$ 乘起来需要多少次乘法?” 如果用状态 $f(i, j)$ 表示这个子问题的值, 不难列出如下的状态转移方程:

$$f(i, j) = \min \{f(i, k) + f(k+1, j) + p_{i-1} p_k p_j\}$$

边界为 $f(i, i) = 0$ 。上述方程有些特殊: 记忆化搜索固然没问题, 但如果要写成递推, 无论按照 i 还是 j 的递增或递减顺序均不正确。正确的方法是按照 $j-i$ 递增的顺序递推, 因为长区间的值依赖于短区间的值。

最优三角剖分。 对于一个 n 个顶点的凸多边形, 有很多种方法可以对它进行三角剖分(triangulation), 即用 $n-3$ 条互不相交的对角线把凸多边形分成 $n-2$ 个三角形。为每个三角形规定一个权函数 $w(i, j, k)$ (如三角形的周长或 3 个顶点的权和), 求让所有三角形权和最大的方案。

【分析】

本题和最优矩阵链乘问题十分相似, 但存在一个显著不同: 链乘表达式反映了决策过程, 而剖分不反映决策过程。举例来说, 在链乘问题中, 方案 $((A_1 A_2)(A_3(A_4 A_5)))$ 只能是先把序列分成 $A_1 A_2$ 和 $A_3 A_4 A_5$ 两部分, 而对于一个三角剖分, “第一刀”可以是任何一条对角线, 如图 9-9 所示。

如果允许随意切割, 则“半成品”多边形的各个顶点是可以在原多边形中随意选取的, 很难简洁定义成状态, 而“矩阵链乘”就不存在这个问题——无论怎样决策, 面临的子问题一定可以用区间表示。在这样的情况下, 有必要把决策的顺序规范化, 使得在规范的决策顺序下, 任意状态都能用区间表示。

定义 $d(i, j)$ 为子多边形 $i, i+1, \dots, j-1, j$ ($i < j$) 的最优值, 则边 $i-j$ 在最优解中一定对应一个三角形 $i-j-k$ ($i < k < j$), 如图 9-10 所示(注意顶点是按照逆时针编号的)。

因此, 状态转移方程为:

$$d(i, j) = \max \{d(i, k) + d(k, j) + w(i, j, k) \mid i < k < j\}$$

时间复杂度为 $O(n^3)$, 边界为 $d(i, i+1) = 0$, 原问题的解为 $d(0, n-1)$ 。

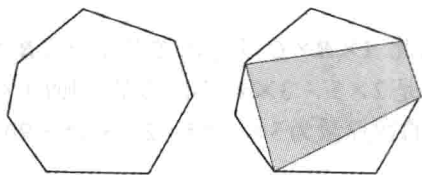


图 9-9 难以简洁表示的状态

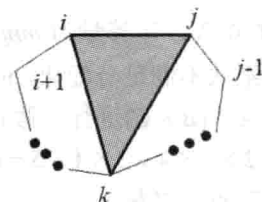


图 9-10 定义的子多边形

例题 9-9 切木棍 (Cutting Sticks, UVa 10003)

有一根长度为 L ($L < 1000$) 的棍子，还有 n ($n < 50$) 个切割点的位置（按照从小到大排列）。你的任务是在这些切割点的位置处把棍子切成 $n+1$ 部分，使得总切割费用最小。每次切割的费用等于被切割的木棍长度。例如， $L=10$ ，切割点为 2, 4, 7。如果按照 2, 4, 7 的顺序，费用为 $10+8+6=24$ ，如果按照 4, 2, 7 的顺序，费用为 $10+4+6=20$ 。

【分析】

设 $d(i, j)$ 为切割小木棍 $i \sim j$ 的最优费用，则 $d(i, j) = \min\{d(i, k) + d(k, j) \mid i < k < j\} + a[j] - a[i]$ ，其中最后一项 $a[j] - a[i]$ 代表第一刀的费用。切完之后，小木棍变成 $i \sim k$ 和 $k \sim j$ 两部分，状态转移方程由此可得。把切割点编号为 $1 \sim n$ ，左边界编号为 0，右边界编号为 $n+1$ ，则答案为 $d(0, n+1)$ 。

状态有 $O(n^2)$ 个，每个状态的决策有 $O(n)$ 个，时间复杂度为 $O(n^3)$ 。值得一提的是，本题可以用四边形不等式优化到 $O(n^2)$ ，有兴趣的读者请参见本书的配套《算法竞赛入门经典——训练指南》或其他参考资料。

例题 9-10 括号序列 (Brackets Sequence, NEERC 2001, UVa1626)

定义如下正规括号序列（字符串）：

- 空序列是正规括号序列。
- 如果 S 是正规括号序列，那么 (S) 和 $[S]$ 也是正规括号序列。
- 如果 A 和 B 都是正规括号序列，那么 AB 也是正规括号序列。

例如，下面的字符串都是正规括号序列：(), [], (()), ([]), ()[], ()[()], 而如下字符串则不是正规括号序列：(, [,],), ([(),

输入一个长度不超过 100 的，由“(”、“)”、“[”、“]”构成的序列，添加尽量少的括号，得到一个规则序列。如有多解，输出任意一个序列即可。

【分析】

设串 S 至少需要增加 $d(S)$ 个括号，转移如下：

- 如果 S 形如 (S') 或者 $[S']$ ，转移到 $d(S')$ 。
- 如果 S 至少有两个字符，则可以分成 AB ，转移到 $d(A) + d(B)$ 。

边界是： S 为空时 $d(S)=0$ ， S 为单字符时 $d(S)=1$ 。注意 $(S'$, $[S'$, $)S'$ 之类全部属于第二种转移，不需要单独处理。

注意：不管 S 是否满足第一条，都要尝试第二种转移，否则 “[]” 会转移到 “[]”，然后就只能加两个括号了。

当然，上述“方程”只是概念上的，落实到程序时要改成子串在原串中的起始点下标，即用 $d(i, j)$ 表示子串 $S[i \sim j]$ 至少需要添加几个括号。下面是递推写法，比记忆化写法要快好几

倍, 而且代码更短。请读者注意状态的枚举顺序:

```
void dp() {
    for(int i = 0; i < n; i++) {
        d[i+1][i] = 0;
        d[i][i] = 1;
    }
    for(int i = n-2; i >= 0; i--)
        for(int j = i+1; j < n; j++) {
            d[i][j] = n;
            if(match(S[i], S[j])) d[i][j] = min(d[i][j], d[i+1][j-1]);
            for(int k = i; k < j; k++)
                d[i][j] = min(d[i][j], d[i][k] + d[k+1][j]);
        }
}
```

本题需要打印解, 但是上面的代码只计算了 d 数组, 如何打印解呢? 可以在打印时重新检查一下哪个决策最好。这样做的好处是节约空间, 坏处是打印时代码较复杂, 速度稍慢, 但是基本上可以忽略不计, 因为只有少数状态需要打印)。

```
void print(int i, int j) {
    if(i > j) return;
    if(i == j) {
        if(S[i] == '(' || S[i] == ')') printf("(");
        else printf("[");
        return;
    }
    int ans = d[i][j];
    if(match(S[i], S[j]) && ans == d[i+1][j-1]) {
        printf("%c", S[i]); print(i+1, j-1); printf("%c", S[j]);
        return;
    }
    for(int k = i; k < j; k++)
        if(ans == d[i][k] + d[k+1][j]) {
            print(i, k); print(k+1, j);
            return;
        }
}
```

本题唯一的陷阱是: 输入串可能是空串, 因此不能用 `scanf("%s", s)` 的方式输入, 只能用 `getchar`、`fgets` 或者 `getline`。

例题 9-11 最大面积最小的三角剖分 (Minimax Triangulation, ACM/ICPC NWERC 2004, UVa1331)

三角剖分是指用不相交的对角线把一个多边形分成若干个三角形。如图 9-11 所示是一

个六边形的几种不同的三角剖分。

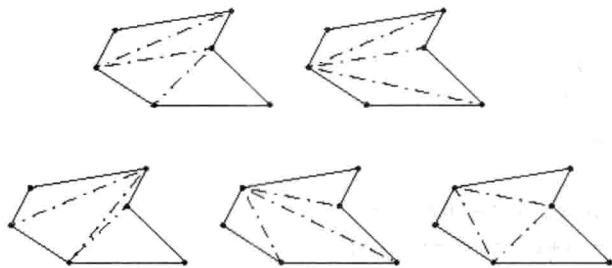


图 9-11 六边形的不同三角剖分

输入一个简单 m ($2 < m < 50$) 边形，找一个最大三角形面积最小的三角剖分。输出最大三角形的面积。在图 9-11 的 5 个方案中，最左边（即左下角）的方案最优。

【分析】

本题的程序实现要用到一些计算几何的知识，不过基本思想是清晰的：首先考虑凸多边形的简单情况。和“最优三角剖分”一样，设 $d(i, j)$ 为子多边形 $i, i+1, \dots, j-1, j$ ($i < j$) 的最优解，则状态转移方程为 $d(i, j) = \min\{S(i, j, k), d(i, k), d(k, j) \mid i < k < j\}$ ，其中 $S(i, j, k)$ 为三角形 $i-j-k$ 的面积。

回到原题。需要保证边 $i-j$ 是对角线^①（唯一的例外是 $i=0$ 且 $j=n-1$ ），具体方法是当边 $i-j$ 不满足条件时直接设 $d(i, j)$ 为无穷大，其他部分和凸多边形的情形完全一样。

9.4.2 树上的动态规划

树的最大独立集。对于一棵 n 个结点的无根树，选出尽量多的结点，使得任何两个结点均不相邻（称为最大独立集），然后输入 $n-1$ 条无向边，输出一个最大独立集（如果有多个解，则任意输出一组）。

【分析】

用 $d(i)$ 表示以 i 为根结点的子树的最大独立集大小。此时需要注意的是，本题的树是无根的：没有所谓的“父子”关系，而只有一些无向边。没关系，只要任选一个根 r ，无根树就变成了有根树，上述状态定义也就有意义了。

结点 i 只有两种决策：选和不选。如果不选 i ，则问题转化为了求出 i 的所有儿子的 d 值再相加；如果选 i ，则它的儿子全部不能选，问题转化为了求出 i 的所有孙子的 d 值之和。换句话说，状态转移方程为：

$$d(i) = \max\{1 + \sum_{j \in gs(i)} d(j), \sum_{j \in s(i)} d(j)\}$$

其中， $gs(i)$ 和 $s(i)$ 分别为 i 的孙子集合与儿子集合，如图 9-12 所示。

代码应如何编写呢？上面的方程涉及“枚举结点 i 的所有儿子和所有孙子”，颇为不便。其实可以换一个角度来看：不从 i 找 $s(i)$ 和 $gs(i)$ 的元素，而从 $s(i)$ 和 $gs(i)$ 的元素找 i 。换句话说，当计算出一个 $d(i)$ 后，用它去更新 i 的父亲和祖父结点的累加值 $\sum_{j \in gs(i)} d(j)$ 和 $\sum_{j \in s(i)} d(j)$ 。

^① 如何判断 $i-j$ 是否为多边形的对角线？限于篇幅，本书没有对计算几何进行专门讨论，请读者参考《算法竞赛入门经典——训练指南》的几何部分。

这样一来，每个结点甚至不必记录其子结点有哪些，只需记录父结点即可。这就是前面提过的“刷表法”。不过这个问题还有另外一种解法，在实践中更加常用，将在例题部分介绍。

树的重心（质心）。对于一棵 n 个结点的无根树，找到一个点，使得把树变成以该点为根的有根树时，最大子树的结点数最小。换句话说，删除这个点后最大连通块（一定是树）的结点数最小。

【分析】

和树的最大独立集问题类似，先任选一个结点作为根，把无根树变成有根树，然后设 $d(i)$ 表示以 i 为根的子树的结点数。不难发现 $d(i) = \sum_{j \in s(i)} d(j) + 1$ 。程序实现也很简单：只需要一次 DFS，在无根树转有根树的同时计算即可，连记忆化都不需要——因为本来就没有重复计算。

那么，删除结点 i 后，最大的连通块有多少个结点呢？结点 i 的子树中最大的有 $\max\{d(j)\}$ 个结点， i 的“上方子树”中有 $n-d(i)$ 个结点，如图 9-13 所示。这样，在动态规划的过程中就可以顺便找出树的重心了。

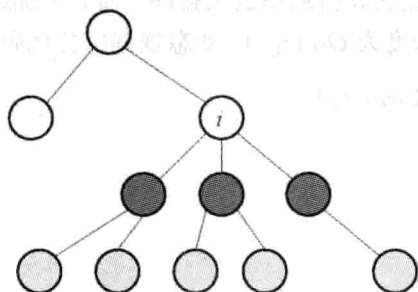


图 9-12 结点 i 的 $gs(i)$ （浅灰色）和 $s(i)$ （深灰色）

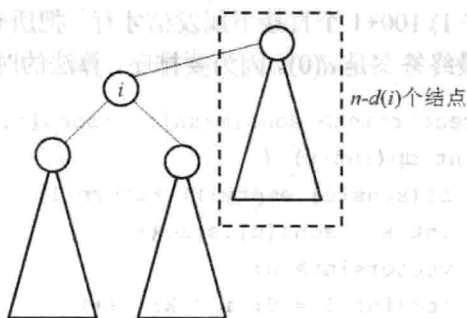


图 9-13 树中的结点分布

树的最长路径（最远点对）。对于一棵 n 个结点的无根树，找到一条最长路径。换句话说，要找到两个点，使得它们的距离最远。

【分析】

和树的重心问题一样，先把无根树转成有根树。对于任意结点 i ，经过 i 的最长路就是连接 i 的两棵不同子树 u 和 v 的最深叶子的路径，如图 9-14 所示。

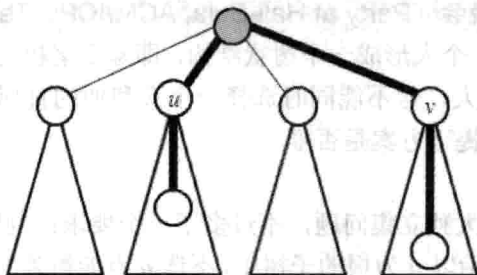


图 9-14 子树 u 和 v 的最深叶子路径

设 $d(i)$ 表示根为结点 i 的子树中根到叶子的最大距离，不难写出状态转移方程：

$d(i) = \max\{d(j)\} + 1$ 。对于每个结点 i , 把所有子结点的 $d(j)$ 都求出来之后, 设 d 值前两大

的结点为 u 和 v , 则 $d(u)+d(v)+2$ 就是所求。

本题还有一个不用动态规划的解法: 随便找一个结点 u , 用 DFS 求出 u 的最远结点 v , 然后再用一次 DFS 求出 v 的最远结点 w , 则 $v-w$ 就是最长路径。

结合上述两个问题的解法, 可以解决下面的问题: 对于一棵 n 个结点的无根树, 求出每个结点的最远点, 要求时间复杂度为 $O(n)$ 。这个问题留给读者思考。

例题 9-12 工人的请愿书 (Another Crisis, UVa 12186)

某公司里有一个老板和 n ($n \leq 10^5$) 个员工组成树状结构, 除了老板之外每个员工都有唯一的直属上司。老板的编号为 0, 员工编号为 $1 \sim n$ 。工人们 (即没有直接下属的员工) 打算签署一项请愿书递给老板, 但是不能跨级递, 只能递给直属上司。当一个中级员工 (不是工人的员工) 的直属下属中不小于 $T\%$ 的人签字时, 他也会签字并且递给他的直属上司。问: 要让公司老板收到请愿书, 至少需要多少个工人签字?

【分析】

设 $d(u)$ 表示让 u 给上级发信最少需要多少个工人。假设 u 有 k 个子结点, 则至少需要 $c = (kT - 1) / 100 + 1$ 个直接下属发信才行。把所有子结点的 d 值从小到大排序, 前 c 个加起来即可。最终答案是 $d(0)$ 。因为要排序, 算法的时间复杂度为 $O(n \log n)$ 。动态规划部分代码如下:

```
vector<int> sons[maxn]; //sons[i] 为结点 i 的子列表
int dp(int u) {
    if(sons[u].empty()) return 1;
    int k = sons[u].size();
    vector<int> d;
    for(int i = 0; i < k; i++)
        d.push_back(dp(sons[u][i]));
    sort(d.begin(), d.end());
    int c = (k*T - 1) / 100 + 1;
    int ans = 0;
    for(int i = 0; i < c; i++) ans += d[i];
    return ans;
}
```

例题 9-13 Hali-Bula 的晚会 (Party at Hali-Bula, ACM/ICPC Tehran 2006, UVa1220)

公司里有 n ($n \leq 200$) 个人形成一个树状结构, 即除了老板之外每个员工都有唯一的直属上司。要求选尽量多的人, 但不能同时选择一个人和他的直属上司。问: 最多能选多少人, 以及在人数最多的前提下方案是否唯一。

【分析】

本题几乎就是树的最大独立集问题, 不过多了一个要求: 判断唯一性。设:

- $d(u,0)$ 和 $f(u,0)$ 表示以 u 为根的子树中, 不选 u 点能得到的最大人数以及方案唯一性 ($f(u,0)=1$ 表示唯一, 0 表示不唯一)。
- $d(u,1)$ 和 $f(u,1)$ 表示以 u 为根的子树中, 选 u 点能得到的最大人数以及方案唯一性。

相应地, 状态转移方程也有两套。

- $d(u,1)$ 的计算: 因为选了 u , 所以 u 的子结点都不能选, 因此 $d(u,1) = \sum\{d(v,0) \mid v \text{ 是 } u \text{ 的子结点}\}$ 。当且仅当所有 $f(v,0)=1$ 时 $f(u,1)$ 才是 1。
- $d(u,0)$ 的计算: 因为 u 没有选, 所以每个子结点 v 可选可不选, 即 $d(u,0) = \sum\{\max(d(v,0), d(v,1))\}$ 。什么情况下方案是唯一的呢? 首先, 如果某个 $d(v,0)$ 和 $d(v,1)$ 相等, 则不唯一; 其次, 如果 $\max$ 取到的那个值对应的 $f=0$, 方案也不唯一 (如 $d(v,0) > d(v,1)$ 且 $f(v,0)=0$, 则 $f(u,0)=0$)。

例题 9-14 完美的服务 (Perfect Service, ACM/ICPC Kaoshiung 2006, UVa1218)

有 n ($n \leq 10000$) 台机器形成树状结构。要求在其中一些机器上安装服务器, 使得每台不是服务器的计算机恰好和一台服务器计算机相邻。求服务器的最少数量。如图 9-15 所示, 图 9-15 (a) 是非法的, 因为 4 同时和两台服务器相邻, 而 6 不与任何一台服务器相邻。而图 9-15 (b) 是合法的。

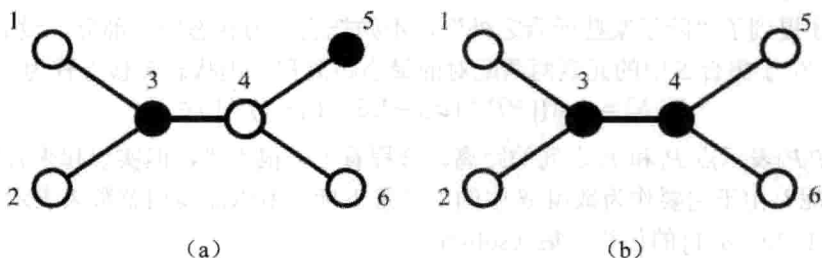


图 9-15 非法与合法的树状结构

【分析】

有了前面的经验, 这次仍然按照每个结点的情况进行分类。

- $d(u,0)$: u 是服务器, 则每个子结点可以是服务器也可以不是。
- $d(u,1)$: u 不是服务器, 但 u 的父亲是服务器, 这意味着 u 的所有子结点都不是服务器。
- $d(u,2)$: u 和 u 的父亲都不是服务器。这意味着 u 恰好有一个儿子是服务器。

状态转移比前面复杂一些, 但也不困难。首先可以写出:

$$d(u,0) = \sum\{\min(d(v,0), d(v,1))\} + 1$$

$$d(u,1) = \sum(d(v,2))$$

而 $d(u,2)$ 稍微复杂一点, 需要枚举当服务器的子结点编号 v , 然后把其他所有子结点 v' 的 $d(v',2)$ 加起来, 再和 $d(v,0)$ 相加。不过如果这样做, 每次枚举 v 都需要 $O(k)$ 时间 (其中 k 是 u 的子结点数目), 而 v 本身要枚举 k 次, 因此计算 $d(u,2)$ 需要花 $O(k^2)$ 时间。

刚才的做法有很多重复计算, 其实可以利用已经算出的 $d(u,1)$ 写出一个新的状态转移方程:

$$d(u,2) = \min(d(u,1) - d(v,2) + d(v,0))$$

这样一来, 计算 $d(u,2)$ 的时间复杂度变为了 $O(k)$ 。因为每个结点只有在计算父亲时被用了 3 次, 总时间复杂度为 $O(n)$ 。

9.4.3 复杂状态的动态规划

最优配对问题。空间里有 n 个点 $P_0, P_1, \dots, P_{n-1}$, 你的任务是把它们配成 $n/2$ 对 (n 是偶数), 使得每个点恰好在一个点对中。所有点对中两点的距离之和应尽量小。 $n \leq 20$, $|x_i|, |y_i|, |z_i| \leq 10000$ 。

【分析】

既然每个点都要配对, 很容易把问题看成如下的多阶段决策过程: 先确定 P_0 和谁配对, 然后是 P_1 , 接下来是 P_2 , …… , 最后是 P_{n-1} 。按照前面的思路, 设 $d(i)$ 表示把前 i 个点两两配对的最小距离和, 然后考虑第 i 个点的决策——它和谁配对呢? 假设它和点 j 配对 ($j < i$), 那么接下来的问题应是“把前 $i-1$ 个点中除了 j 之外的其他点两两配对”, 它显然无法用任何一个 d 值来刻画——此处的状态定义无法体现出“除了一些点之外”这样的限制。

当发现状态无法转移后, 常见的方法是增加维度, 即增加新的因素, 更细致地描述状态。既然刚才提到了“除了某些元素之外”, 不妨把它作为状态的一部分, 设 $d(i, S)$ 表示把前 i 个点中, 位于集合 S 中的元素两两配对的最小距离和, 则状态转移方程为:

$$d(i, S) = \min \{ |P_i P_j| + d(i-1, S - \{i\} - \{j\}) \mid j \in S \}$$

其中, $|P_i P_j|$ 表示点 P_i 和 P_j 之间的距离。方程看上去很不错, 但实现起来有问题: 如何表示集合 S 呢? 由于它要作为数组 d 中的第二维下标, 所以需要整数来表示集合, 确切地说, 是 $\{0, 1, 2, \dots, n-1\}$ 的任意子集 (subset)。

在第7章的“子集枚举”部分, 曾介绍过子集的二进制表示, 现在再次用到此知识:

```
for(int i = 0; i < n; i++)
    for(int S = 0; S < (1<<n); S++) {
        d[i][S] = INF;
        for(int j = 0; j < i; j++) if(S & (1<<j))
            d[i][S] = max(d[i][S], dist(i, j) + d[i-1][S^(1<<i)^(1<<j)]);
    }
```

上述程序中故意用了很多括号, 传达给读者的信息是: 位运算的优先级低, 初学者很容易弄错。例如, “ $1<<n-1$ ”的正确解释是“ $1<<(n-1)$ ”, 因为减法的优先级比左移要高。为了保险起见, 应多用括号。另一个技巧是利用C语言中“0为假, 非0为真”的规定简化表达式: “ $\text{if}(S \& (1<<j))$ ”的实际含义是“ $\text{if}((S \& (1<<j)) != 0)$ ”。

提示 9-19: 位运算的优先级往往比较低。如果不确定表达式的计算顺序, 应多用括号。

由于大量使用了形如 $1<<n$ 的表达式, 此类表达式中, 左移运算符“ $<<$ ”的含义是“把各个位往左移动, 右边补0”。根据二进制运算法则, 每次左移一位就相当于乘以2, 因此 $a<<b$ 相当于 $a*2^b$, 而在集合表示法中, $1<<i$ 代表单元素集合 $\{i\}$ 。由于0表示空集, “ $S \& (1<<j)$ ”不等于0就意味着“ S 和 $\{j\}$ 的交集不为空”。

上面的方程可以进一步简化。事实上, 阶段 i 根本不用保存, 它已经隐含在 S 中了—— S 中的最大元素就是 i 。这样, 可直接用 $d(S)$ 表示“把 S 中的元素两两配对的最小距离和”,

则状态转移方程为:

$$d(S) = \min\{|P_i P_j| + d(S - \{i\} - \{j\}) \mid j \in S, i = \max\{S\}\}$$

状态有 2^n 个, 每个状态有 $O(n)$ 种转移方式, 总时间复杂度为 $O(n2^n)$ 。

提示 9-20: 如果用二进制表示子集并进行动态规划, 集合中的元素就隐含了阶段信息。例如, 可以把集合中的最大元素想象成“阶段”。

值得一提的是, 不少用户一直在用这样的状态转移方程:

$$d(S) = \min\{|P_i P_j| + d(S - \{i\} - \{j\}) \mid i, j \in S\}$$

它和刚才的方程很类似, 唯一的不同是: i 和 j 都是需要枚举的。这样做虽然也没错, 但每个状态的转移次数高达 $O(n^2)$, 总时间复杂度为 $O(n^2 2^n)$, 比刚才的方法慢。这个例子再次说明: 即使用相同的状态描述, 减少决策也是很重要的。

提示 9-21: 即使状态定义相同, 过多地考虑不必要的决策仍可能会导致时间复杂度上升。

接下来出现了一个新问题: 如何求出 S 中的最大元素呢? 用一个循环判断即可。当 S 取遍 $\{0, 1, 2, \dots, n-1\}$ 的所有子集时, 平均判断次数仅为 2 (想一想, 为什么)。

```
for(int S = 0; S < (1<<n); S++) {
    int i, j;
    d[S] = INF;
    for(i = 0; i < n; i++)
        if(S & (1<<i)) break;
    for(j = i+1; j < n; j++)
        if(S & (1<<j)) d[S] = max(d[S], dist(i, j) + d[S^(1<<i)^(1<<j)]);
}
```

注意, 在上述的程序中求出的 i 是 S 中的最小元素, 而不是最大元素, 但这并不影响答案。另外, j 的枚举只需从 $i+1$ 开始——既然 i 是 S 中的最小元素, 则说明其他元素自然均比 i 大。最后需要说明的是 S 的枚举顺序。不难发现: 如果 S' 是 S 的真子集, 则一定有 $S' < S$, 因此若以 S 递增的顺序计算, 需要用到某个 d 值时, 它一定已经计算出来了。

提示 9-22: 如果 S' 是 S 的真子集, 则一定有 $S' < S$ 。在用递推法实现子集的动态规划时, 该规则往往可以确定计算顺序。

货郎担问题 (TSP)。 有 n 个城市, 两两之间均有道路直接相连。给出每两个城市 i 和 j 之间的道路长度 L_{ij} , 求一条经过每个城市一次且仅一次, 最后回到起点的路线, 使得经过的道路总长度最短。 $N \leq 15$, 城市编号为 $0 \sim n-1$ 。

【分析】

TSP 是一道经典的 NPC 难题^①, 不过因为本题规模小, 可以用动态规划求解。首先注意到可以直接规定起点和终点为城市 0 (想一想, 为什么), 然后设 $d(i, S)$ 表示当前在城市

^① 所谓 NPC, 即 NP-完全问题 (NP-Complete Problem), 是指一类目前还没有找到多项式算法的问题。它的确切定义超出了本书的范围。

i ，还需访问集合 S 中的城市各一次后回到城市 0 的最短长度，则

$$d(i, S) = \min\{d(j, S - \{j\}) + \text{dist}(i, j) \mid j \in S\}$$

边界为 $d(i, \{\}) = \text{dist}(0, i)$ 。最终答案是 $d(0, \{1, 2, 3, \dots, n-1\})$ ，时间复杂度为 $O(n^2 2^n)$ 。

图的色数。图论有一个经典问题是这样的：给一个无向图 G ，把图中的结点染成尽量少的颜色，使得相邻结点颜色不同。

【分析】

设 $d(S)$ 表示把结点集 S 染色，所需要颜色数的最小值，则 $d(S) = d(S - S') + 1$ ，其中 S' 是 S 的子集，并且内部没有边（即不存在 S' 内的两个结点 u 和 v 使得 u 和 v 相邻）。换句话说， S' 是一个“可以染成同一种颜色”的结点集。

首先通过预处理保存每个结点集是否可以染成同一种颜色（即“内部没有边”），则算法的主要时间取决于“高效的枚举一个集合 S 的所有子集”。

如何枚举 S 的子集呢？详见下面的代码（代码中的 S_0 就是上面的 S' ）：

```
d[0] = 0;
for(int S = 1; S < (1<<n); S++) {
    d[S] = INF;
    for(int S0 = S; S0; S0 = (S0-1)&S)
        if(no_edges_inside[S0]) d[S] = min(d[S], d[S-S0]+1);
}
```

如何分析上述算法的时间复杂度？它等于全集 $\{1, 2, \dots, n\}$ 的所有子集的“子集个数”之和。如果不好理解，可以令 $c(S)$ 表示集 S 的子集的个数（它等于 $2^{|S|}$ ），则本题的时间复杂度为 $\sum\{c(S_0) \mid S_0 \text{ 是 } \{1, 2, 3, \dots, n\} \text{ 的子集}\}$ 。元素个数相同的集合，其子集个数也相同，可以按照元素个数“合并同类项”。元素个数为 k 的集合有 $C(n, k)$ 个，其中每个集合有 2^k 个子集，因此本题的时间复杂度为 $\sum\{C(n, k)2^k\} = (2+1)^n = 3^n$ ，其中第一个等号用到了第 10 章即将学到的二项式定理（不过是“反着”用的）。

提示 9-23：枚举 $1 \sim n$ 的每个集合 S 的所有子集的总时间复杂度为 $O(3^n)$ 。

例题 9-15 校长的烦恼 (Headmaster's Headache, UVa 10817)

某校有 m 个教师和 n 个求职者，需讲授 s 个课程 ($1 \leq s \leq 8, 1 \leq m \leq 20, 1 \leq n \leq 100$)。已知每人的工资 c ($10000 \leq c \leq 50000$) 和能教的课程集合，要求支付最少的工资使得每门课都至少有两名教师能教。在职教师不能辞退。

【分析】

本题的做法有很多。一种相对容易实现的方法是：用两个集合 s_1 表示恰好有一个人教的科目集合， s_2 表示至少有两个人教的科目集合，而 $d(i, s_1, s_2)$ 表示已经考虑了前 i 个人时的最小花费。注意，把所有人一起从 0 编号，则编号 $0 \sim m-1$ 是在职教师， $m \sim m+n-1$ 是应聘者。状态转移方程为 $d(i, s_1, s_2) = \min\{d(i+1, s_1', s_2') + c[i], d(i+1, s_1, s_2)\}$ ，其中第一项表示“聘用”，第二项表示“不聘用”。当 $i \geq m$ 时状态转移方程才出现第二项。这里 s_1' 和 s_2' 分别表示“招聘第 i 个人之后 s_1 和 s_2 的新值”，具体计算方法见代码。

下面代码中的 $st[i]$ 表示第 i 个人能教的科目集合（注意输入中科目从 1 开始编号，而代

码的其他部分中科目从 0 开始编号, 因此输入时要转换一下)。下面的代码用到了一个技巧: 记忆化搜索中有一个参数 s_0 , 表示没有任何人能教的科目集合。这个参数并不需要记忆 (因为有了 s_1 和 s_2 就能算出 s_0), 仅是为了编程的方便 (详见 s_1 和 s_2 的计算方式)。最终结果是 $dp(0, (1 \leq s)-1, 0, 0)$, 因为初始时所有科目都没有人教。

```
int m, n, s, c[maxn], st[maxn], d[maxn][1<<maxs][1<<maxs];
```

```
int dp(int i, int s0, int s1, int s2) {
```

```
    if(i == m+n) return s2 == (1<<s) - 1 ? 0 : INF;
```

```
    int& ans = d[i][s1][s2];
```

```
    if(ans >= 0) return ans;
```

```
    ans = INF;
```

```
    if(i >= m) ans = dp(i+1, s0, s1, s2); //不选
```

```
    int m0 = st[i] & s0, m1 = st[i] & s1;
```

```
    s0 ^= m0; s1 = (s1 ^ m1) | m0; s2 |= m1;
```

```
    ans = min(ans, c[i] + dp(i+1, s0, s1, s2)); //选
```

```
    return ans;
```

```
}
```

本题还有其他解法, 例如, 分别用 0, 1, 2 表示每个科目是没人教、恰好一个人教和至少两个人教, 这样就可以用一个三进制数来保存状态, 而不是两个集合。不过这样做编程稍微麻烦一些, 而且时间效率差不多 (在上面的代码中, 虽然 d 数组有 4^s 个元素, 但因为记忆化的关系, 只用到了 3^s 个)。

例题 9-16 20 个问题 (Twenty Questions, ACM/ICPC Tokyo 2009, UVa1252)

有 n ($n \leq 128$) 个物体, m ($m \leq 11$) 个特征。每个物体用一个 m 位 01 串表示, 表示每个特征是具备还是不具备。我在心里想一个物体 (一定是这 n 个物体之一), 由你来猜。

你每次可以询问一个特征, 然后我会告诉你: 我心里的物体是否具备这个特征。当你确定答案之后, 就把答案告诉我 (告知答案不算“询问”)。如果你采用最优策略, 最少需要询问几次能保证猜到?

例如, 有两个物体: 1100 和 0110, 只要询问特征 1 或者特征 3, 就能保证猜到。

【分析】

为了叙述方便, 设“心里想的物体”为 W 。首先在读入时把每个物体转化为一个二进制整数。不难发现, 同一个特征不需要问两遍, 所以可以用一个集合 s 表示已经询问的特征集。在这个集合 s 中, 有些特征是 W 所具备的, 剩下的特征是 W 不具备的。用集合 a 来表示“已确认物体 W 具备的特征集”, 则 a 一定是 s 的子集。

设 $d(s, a)$ 表示已经问了特征集 s , 其中已确认 W 所具备的特征集为 a 时, 还需要询问的最小次数。如果下一次提问的对象是特征 k (这就是“决策”), 则询问次数为:

$$\max\{d(s+\{k\}, a+\{k\}), d(s+\{k\}, a)\}+1$$

考虑所有的 k , 取最小值即可。边界条件为: 如果只有一个物体满足“具备集合 a 中的所有特征, 但不具备集合 $s-a$ 中的所有特征”这一条件, 则 $d(s, a)=0$, 因为无须进一步询问,

已经可以得到答案。

因为 a 为 s 的子集, 所以状态总数为 3^m , 时间复杂度为 $O(m \cdot 3^m)$ 。对于每个 s 和 a , 可以先把满足该条件的物体个数统计出来, 保存在 $\text{cnt}[s][a]$, 避免状态转移的时候重复计算。统计 $\text{cnt}[s][a]$ 的方法是枚举 s 和物体, 时间复杂度为 $O(n \cdot 2^m)$, 所以总时间复杂度为 $O(n \cdot 2^m + m \cdot 3^m)$ 。对于本题的规模来说 $O(n \cdot 2^m)$ 可以忽略不计。

例题 9-17 基金管理 (Fund Management, ACM/ICPC NEERC 2007, UVa1412)

你有 c ($0.01 \leq c \leq 10^8$) 美元现金, 但没有股票。给你 m ($1 \leq m \leq 100$) 天时间和 n ($1 \leq n \leq 8$) 支股票供你买卖, 要求最后一天结束后不持有任何股票, 且剩余的钱最多。买股票不能赊账, 只能用现金买。

已知每只股票每天的价格 ($0.01 \sim 999.99$, 单位是美元/股) 与参数 s_i 和 k_i , 表示一手股票是 s_i ($1 \leq s_i \leq 10^6$) 股, 且每天持有的手数不能超过 k_i ($1 \leq k_i \leq k$), 其中 k 为每天持有的总手数上限。每天要么不操作, 要么选一只股票, 买或卖它的一手股票。 c 和股价均最多包含两位小数 (即美分)。最优解保证不超过 10^9 。要求输出每一天的决策 (HOLD 表示不变, SELL 表示卖, BUY 表示买)。

【分析】

根据前面的经验, 可以用 $d(i, p)$ 表示经过 i 天之后, 资产组合为 p 时的现金的最大值。其中 p 是一个 n 元组, $p_i \leq k_i$ 表示第 i 只股票有 p_i 手。根据题目规定, $p_1 + \dots + p_n \leq k$ 。因为 $0 \leq p_i \leq 8$, 理论上最多只有 $9^8 < 5 \cdot 10^7$ 种可能, 所以可以用一个九进制整数来表示 p 。

一共有 3 种决策: HOLD、BUY 和 SELL, 分别进行转移即可。注意在考虑购买股票时不要忘记判断当前拥有的现金是否足够。细心的读者可能已经发现: 正因为如此, 本题并不是一个标准的 DAG 最长/短路问题, 因为某些边 $u \rightarrow v$ 的存在性依赖于起点到 u 的最短路值。也就是说, 本题的状态不能像之前的 DAG 问题一样“反着定义”: 如果用 $d(i, p)$ 表示资产组合为 p , 从第 i 天开始到最后能拥有的现金的最大值, 就没法转移了 (想一想, 为什么)。

这样的做法虽然不错^①, 但是效率却不够高, 因为九进制整数无法直接进行“买卖股票”的操作, 需要解码成 n 元组才行。因为几乎每次状态转移都会涉及编码、解码操作, 状态转移的时间大幅度提升, 最终导致超时。

解决方法是事先计算出所有可能的状态并且编号 (还记得第 5 章中的“集合栈计算机”吗?), 代码如下:

```
vector<vector<int>> > states;
map<vector<int>, int> ID;

void dfs(int stock, vector<int>& lots, int totlot) {
    if (stock == n) {
        ID[lots] = states.size();
        states.push_back(lots);
    }
    else for (int i = 0; i <= k[stock] && totlot + i <= kk; i++) {
```

^① 完整实现见代码仓库。

```

    lots[stock] = i;
    dfs(stock+1, lots, totlot + i);
}
}

```

然后构造一个状态转移表，用 `buy_next[s][i]` 和 `sell_next[s][i]` 分别表示状态 `s` 进行“买股票 `i`”和“卖股票 `i`”之后转移到的状态编号，代码如下：

```
int buy_next[maxstate][maxn], sell_next[maxstate][maxn];
```

```
void init() {
```

```
    vector<int> lots(n);
```

```
    states.clear();
```

```
    ID.clear();
```

```
    dfs(0, lots, 0);
```

```
    for(int s = 0; s < states.size(); s++) {
```

```
        int totlot = 0;
```

```
        for(int i = 0; i < n; i++) totlot += states[s][i];
```

```
        for(int i = 0; i < n; i++) {
```

```
            buy_next[s][i] = sell_next[s][i] = -1;
```

```
            if(states[s][i] < k[i] && totlot < kk) {
```

```
                vector<int> newstate = states[s];
```

```
                newstate[i]++;
```

```
                buy_next[s][i] = ID[newstate];
```

```
            }
```

```
            if(states[s][i] > 0) {
```

```
                vector<int> newstate = states[s];
```

```
                newstate[i]--;
```

```
                sell_next[s][i] = ID[newstate];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

动态规划主程序采用刷表法（读者也可以试着改成倒推的填表法），为了方便起见，另外编写了“更新状态”的函数 `update`，读者可以自行体会它的好处。为了打印解，在更新解 `d` 时还要更新最优策略 `opt` 和“上一个状态” `prev`。注意下面的 `price[i][day]` 表示第 `day` 天时一手股票 `i` 的价格，而不是输入中的“每股价格”。

```
double d[maxm][maxstate];
```

```
int opt[maxm][maxstate], prev[maxm][maxstate];
```

```
void update(int day, int s, int s2, double v, int o) {
```

```
    if(v > d[day+1][s2]) {
```